

Machine learning in finance :

Theoretical foundations

Huyên PHAM

Ecole Polytechnique, CMAP

<https://sites.google.com/site/phamxuanhuyen/>

Email : huyen.pham@polytechnique.edu or pham@ensae.fr

Master M2MO, SFA

Mastère spécialisé FGR, DS

ENSAE 3A

Year 2025-2026

Administrative information

- When and where

- Wednesday : 15h15-16h45, 17h-18h30, feb 18, 25, march 11, 18 2026
- Location : ENSAE, Amphi 250 or 200

- Lecture notes

- Slides will be posted on pamplemousse

- Evaluation

- Project by groups of three
- List of projects are posted on pamplemousse
- Choices have to be made before March 21
- Report (no more than 10 pages), Deadline, April 25
- Oral defense : between April 26 and may 8.

Course overview

- Introduction to some **fundamental concepts** of machine learning :
 - Gaussian process regression
 - Deep learning and artificial neural networks
 - Stochastic approximation
 - PDEs and Backward Stochastic Differential Equations
 - Generative modeling
 - Markov decision process and dynamic programming
 - Deep reinforcement learning, policy gradient
- **Applications in finance** :
 - Deep hedging and portfolio optimization
 - Market generators
 - Computational methods in high dimension
 - Optimal trading

Transformation of Quantitative Finance

Quantitative finance has undergone a major transformation :

- 1990s–2000s : analytical models and PDE methods
- 2000s–2010s : Monte Carlo simulation and numerical schemes
- 2015–today : machine learning and data-driven approaches

This evolution is driven by :

- increasing model complexity
- high-dimensional problems
- real-time computational constraints
- availability of large-scale data and simulations

From Classical Quantitative Finance to Machine Learning

Classical paradigm :

- start from economic assumptions
- specify a stochastic model
- derive prices, hedges, or strategies via PDEs or Monte Carlo

Machine learning perspective :

- treat pricing, calibration, and hedging as learning problems
- learn functional relationships from data or simulations
- replace expensive solvers by surrogate models

Machine learning complements, rather than replaces, financial modeling.

Four Roles of Machine Learning in Finance

Machine learning plays four major roles in modern quantitative finance :

- ① Function approximation
- ② Optimization under uncertainty
- ③ Sequential decision making
- ④ Generative modeling

Each lecture focuses on one of these paradigms.

Machine Learning Paradigms in Finance

Machine Learning in Quantitative Finance

Gaussian Processes

Surrogate Modeling

Pricing, Greeks, calibration

Reinforcement Learning

Sequential Decision Making

Trading, execution, portfolio control

Deep Learning

High-Dimensional Approximation

Stochastic control, deep hedging

Generative Models

Synthetic Market Environments

Market simulation, scenario generation

Four complementary paradigms for modern quantitative finance

Course structure

- ① Gaussian processes and financial applications
- ② Neural networks for high-dimensional problems
 - Deep hedging
 - Stochastic control
 - Numerical resolution of PDEs
- ③ (Deep) Reinforcement learning and applications
 - Q-learning, policy gradient methods
 - Applications in trading and market making
- ④ Generative modeling and market generators
 - GANs, diffusion models, Schrödinger bridge

Why Start with Gaussian Processes ?

Gaussian processes provide :

- A fully Bayesian approach to function approximation
- Closed-form uncertainty quantification
- A bridge between statistics and deep learning
- A foundation for Bayesian optimization

They illustrate the core ideas that reappear in deep learning, reinforcement learning and generative modeling.

Lecture 1 : Gaussian process regression and financial applications

- 1 A primer on Gaussian process
 - Prediction
 - Bayesian optimization
- 2 Applications of GPR in finance
 - Applications to fast pricing and and fitting
 - GP surrogates for Delta Hedging
 - Other applications

Outline

- 1 A primer on Gaussian process
 - Prediction
 - Bayesian optimization
- 2 Applications of GPR in finance
 - Applications to fast pricing and and fitting
 - GP surrogates for Delta Hedging
 - Other applications

Gaussian Processes : Historical Perspective

- **Kolmogorov (1940s)** Probabilistic foundations of stochastic processes
(Gaussian processes as collections of consistent finite-dimensional Gaussians)
- **Wahba (1970s–1990s)** Spline smoothing and regularization theory
(e.g. yield curve smoothing, volatility smile interpolation)
- **Stein (1990s)** Spatial statistics and covariance modeling
(Robust prediction, Matérn kernels, model misspecification)
- **Williams & Rasmussen (1997–2006)** Gaussian processes as Bayesian non-parametric models for machine learning

Key insight : Gaussian processes unify deterministic regularization and Bayesian modeling.

Prediction Problem

We observe noisy data :

$$y_i = f(x_i) + \varepsilon_i, \quad i = 1, \dots, n,$$

where f is an unknown function, ε_i are i.i.d. noise terms.

Example (finance) :

- x_i = strike or maturity
- y_i = option price or implied volatility
- f = unknown pricing surface

Objective : Given observations (x_i, y_i) , estimate the value of $f(x^*)$ at a new input x^* , *together with a measure of uncertainty.*

Key questions :

- How to interpolate / smooth noisy data ?
- How to quantify uncertainty away from observed points ?
- How to incorporate prior beliefs on regularity of f ?

From Parametric to Non-Parametric Regression

Classical approach : Assume a parametric model

$$f(x) = \beta_0 + \beta_1 x + \dots$$

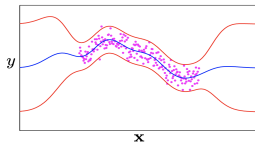
and estimate parameters β .

Limitation :

- Model structure fixed in advance
- Risk of underfitting or misspecification

Non-parametric approach :

Instead of specifying a functional form, we model *a distribution over functions* :



Data update our prior belief on f to obtain a posterior distribution.

GP = Bayesian non-parametric regression

What is a Gaussian Process?

Definition. A Gaussian process (GP) is a stochastic process

$$\{f(x) : x \in \mathcal{X}\}$$

such that for any finite collection of points $x_1, \dots, x_n \in \mathcal{X}$, the random vector

$$(f(x_1), \dots, f(x_n))$$

is multivariate Gaussian.

Key ideas :

- Instead of modeling a single function, we model a random function.
- Extension of Gaussian random vector to Gaussian distribution over functions on continuous domain (time/space)

Characteristics of a GP

A Gaussian process $\{f(x), x \in \mathcal{X}\}$ is fully characterized by its mean function

$$m(x) = \mathbb{E}[f(x)]$$

and the covariance function, called **kernel** :

$$k(x, x') = \text{cov}(f(x), f(x')),$$

and we often write :

$$f \sim \mathcal{GP}(m, k).$$

- GP can be viewed as Gaussian distributions over functions.
- Properties of the random function f are determined by m and kernel k .

Remarks.

- We often assume that $m \equiv 0$. What relates one observation x to another one x' is the covariance $k(x, x')$.
- k must be symmetric and positive semi-definite
- The kernel encodes smoothness and structural assumptions.

Squared Exponential (SE) Kernel (also called Radial Basis Function RBF kernel)

A widely used covariance function is the squared exponential (SE) kernel :

$$k(x, x') = \sigma_f^2 \exp\left(-\frac{\|x - x'\|^2}{2\ell^2}\right)$$

- σ_f^2 : signal variance (amplitude)
- ℓ : length-scale parameter

Interpretation :

- Inputs close to each other are highly correlated
- Correlation decays smoothly with distance
- Larger $\ell \Rightarrow$ smoother functions

Properties :

- Stationary kernel : $k(x, x') = \kappa(x - x')$
- Infinitely differentiable sample paths

Other Covariance Functions

Linear kernel (linear regression) :

$$k(x, x') = \sigma_f^2 + xx'$$

Brownian motion kernel :

$$k(x, x') = \min(x, x'), \quad x, x' \geq 0$$

Periodic kernel :

$$k(x, x') = \sigma_f^2 \exp\left(-\frac{2 \sin^2(\pi|x - x'|/\lambda)}{\ell^2}\right)$$

Interpretation :

- Linear trends and drift
- Random walk-type behavior
- Seasonality and cyclic effects

Constructing new covariances from old

- **Sum** : $k(x, x') = k_1(x, x') + k_2(x, x')$.

Example : two independent random functions f_1 and f_2 with kernels k_1 and k_2 . Used e.g. to add together kernels with different characteristic length-scales.

- **Product** : $k(x, x') = k_1(x, x').k_2(x, x')$.

Example : If observations are defined by couple (t, x) with $t \in \mathbb{R}_+$ the time index, $x \in \mathbb{R}^d$ (e.g. daily prices of multi-assets), one can define a kernel on the whole space-time by

$$k((t, x), (t', x')) = k_{time}(t, t').k_{space}(x, x'),$$

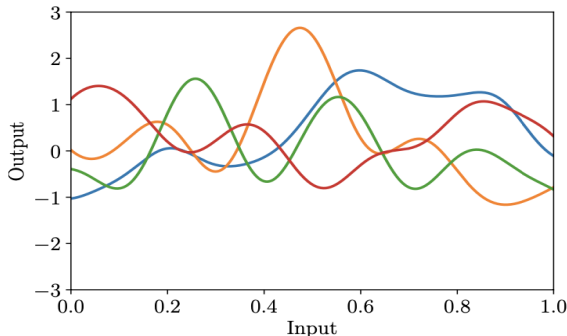
where k_{time} is a kernel function for time patterns, and k_{space} is a kernel function for space patterns.

→ **Composite kernel** : combine multiple base kernels to capture different aspects of data.

Practical example : modeling a stock price that generally trends upwards over time but also fluctuates (market volatility)

- Linear kernel captures overall upward trend
- Gaussian kernel captures short-term fluctuations and volatility

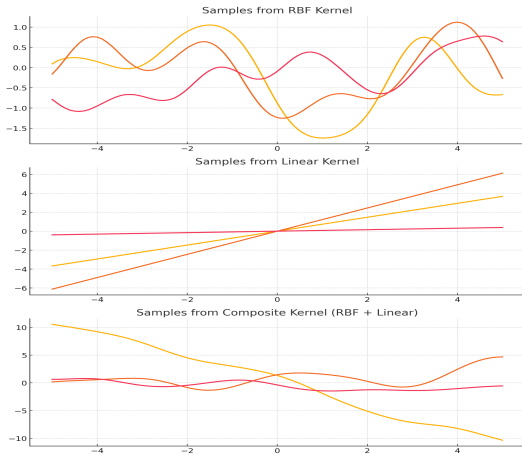
Sample paths of a GP



(a) $\mathcal{K}_{\text{SE}}$

Four sample paths of one-dimensional GP with SE kernel. This suggests smooth underlying function (property of SE kernel). One can generate rougher sample functions with the so-called **Matern** class of kernels.

Samples from composite kernel



Financial Interpretation of Kernels

The kernel encodes assumptions on how financial quantities vary between observed points.

Kernel choice	Financial interpretation
Squared exponential	Smooth pricing surfaces
Matérn	Rough volatility, limited regularity
Linear	Trend, carry, drift effects
Brownian	Random walk-type behavior
Periodic	Seasonality (energy, commodities)

Key message : Choosing a kernel means choosing a financial prior.
Kernel misspecification affects interpretation more than prediction

Example : Designing a Kernel for Option Prices

Consider an option price surface :

$$P = P(t, S)$$

as a function of time t and underlying price S .

Modeling assumptions :

- Prices vary smoothly in S
- Strong dependence for nearby maturities
- Different regularity in time and space

Kernel design :

$$k((t, S), (t', S')) = k_t(t, t') \cdot k_S(S, S')$$

where :

- k_t : time kernel (e.g. Matérn)
- k_S : price kernel (e.g. squared exponential)

This encodes smoothness in S and controlled regularity in t .

Relationship Between Gaussian Processes and Neural Networks

Consider a one-hidden-layer neural network :

$$f(x) = b + \sum_{i=1}^M w_i \sigma(w_i^{(0)} x + b_i^{(0)})$$

Assume :

- weights and biases are i.i.d. random variables
- zero-mean and finite variance

As the number of hidden units $M \rightarrow \infty$, the function f converges in distribution to a Gaussian process.

The corresponding kernel depends on the activation function σ .

Prediction with Gaussian Processes

Gaussian process regression is a Bayesian method.

Step 1 : Prior

Choose a prior distribution over functions :

$$f \sim \mathcal{GP}(m, k)$$

reflecting prior beliefs on regularity and structure.

Step 2 : Conditioning on data

Given observations (x_i, y_i) , update the prior to obtain a posterior distribution over functions.

Outcome :

- A predictive mean (best estimate)
- A predictive variance (uncertainty)

Gaussian Process Regression : Observation Model

We observe noisy data :

$$y_i = f(x_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma_\varepsilon^2), \quad i = 1, \dots, n.$$

Let :

$$\mathbf{x} = (x_1, \dots, x_n), \quad \mathbf{y} = (y_1, \dots, y_n)^\top.$$

Assume prior :

$$f \sim \mathcal{GP}(m, k).$$

Goal : compute the posterior distribution of $f(x^*)$ given the observations $\mathbf{y}$.

Joint Gaussian Structure

From the GP prior and Gaussian noise, the vector of observations and a new value satisfy :

$$\begin{pmatrix} \mathbf{y} \\ f(\mathbf{x}^*) \end{pmatrix} \sim \mathcal{N} \left(\begin{pmatrix} M(\mathbf{x}) \\ m(\mathbf{x}^*) \end{pmatrix}, \begin{pmatrix} K(\mathbf{x}, \mathbf{x}) + \sigma_\varepsilon^2 I_n & K(\mathbf{x}, \mathbf{x}^*) \\ K(\mathbf{x}^*, \mathbf{x}) & k(\mathbf{x}^*, \mathbf{x}^*) \end{pmatrix} \right)$$

where :

$$M(\mathbf{x})_i = m(x_i), \quad K(\mathbf{x}, \mathbf{x})_{ij} = k(x_i, x_j), \quad K(\mathbf{x}, \mathbf{x}^*)_i = k(x_i, \mathbf{x}^*), \quad K(\mathbf{x}^*, \mathbf{x}) = K(\mathbf{x}, \mathbf{x}^*)^\top.$$

Posterior Distribution

Conditioning a Gaussian gives Annex :

$$f(x^*) \mid \mathbf{y} \sim \mathcal{N}(\hat{m}(x^*), \hat{v}(x^*)),$$

with **mean predictor** :

$$\hat{m}(x^*) = m(x^*) + K(x^*, \mathbf{x})(K(\mathbf{x}, \mathbf{x}) + \sigma_\epsilon^2 I)^{-1}(\mathbf{y} - M(\mathbf{x})),$$

and **variance predictive** :

$$\hat{v}(x^*) = k(x^*, x^*) - K(x^*, \mathbf{x})(K(\mathbf{x}, \mathbf{x}) + \sigma_\epsilon^2 I)^{-1}K(\mathbf{x}, x^*).$$

→ Training cost : $\mathcal{O}(n^3)$ (matrix inversion).

Interpretation of the Posterior

The predictive mean :

- Linear combination of observed values
- Weights determined by covariance structure

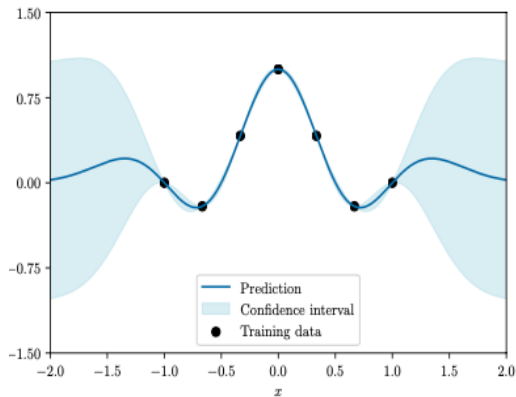
The predictive variance :

- Small near observed points
- Large far from data
- Reflects model uncertainty

Kernel controls how information propagates.

Illustration : posterior distribution of $\text{sinc}(2x)$ with SE kernel

Training set of 7 patterns measured without noise from the cardinal sine function :
 $\text{sinc}(2x) := \frac{\sin(2x)}{2x}$.



Hyperparameter Selection (Marginal Likelihood)

Kernel hyperparameters (e.g. σ_f^2 , ℓ) and noise variance σ_ε^2 control the fit and uncertainty.

Given $\mathbf{y} \mid \mathbf{x} \sim \mathcal{N}(M(\mathbf{x}), K_\theta(\mathbf{x}, \mathbf{x}) + \sigma_\varepsilon^2 I)$, we select θ by maximizing the **log marginal likelihood** :

$$\log p(\mathbf{y} \mid X, \theta) = -\frac{1}{2}(\mathbf{y} - M(\mathbf{x}))^\top C^{-1}(\mathbf{y} - M(\mathbf{x})) - \frac{1}{2} \log |C| - \frac{n}{2} \log(2\pi),$$

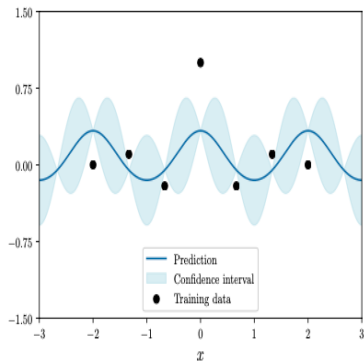
where

$$C = K_\theta(\mathbf{x}, \mathbf{x}) + \sigma_\varepsilon^2 I.$$

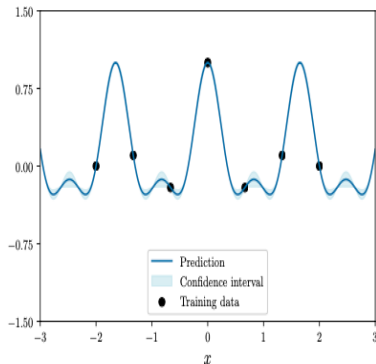
In practice :

- Optimize θ by gradient methods (or quasi-Newton)
- Compute $C^{-1}(\mathbf{y} - M)$ and $\log |C|$ via Cholesky decomposition

Illustration of ML estimation with periodic kernel



(a) Posterior distribution before ML estimation with $\sigma = 1$, $\lambda = 2$, $\ell = 1$, $\sigma_\varepsilon = 10^{-7}$



(b) Posterior distribution after ML estimation : $\hat{\sigma} = 0.7657$, $\hat{\lambda} = 1.6506$, $\hat{\ell} = 0.7664$, $\hat{\sigma}_\varepsilon = 2.46 \times 10^{-7}$

Bayesian Optimization

Goal : minimize an expensive black-box function

$$\min_{x \in \mathcal{D}} f(x)$$

Challenges :

- f is expensive to evaluate (e.g. Monte Carlo, calibration)
- No analytical gradient available
- Limited evaluation budget

Idea :

- Place a GP prior on f
- Use it as a surrogate model
- Sequentially select evaluation points

Bayesian Optimization Algorithm

Initialize with a small set of evaluations :

$$\mathcal{D}_0 = \{(x_i, f(x_i))\}_{i=1}^{n_0}.$$

For $n = 0, 1, \dots$:

- ❶ Fit GP to current data $\mathcal{D}_n$
- ❷ Construct acquisition function $U_n(x)$
- ❸ Select next point :

$$x_{n+1} = \arg \max_x U_n(x)$$

- ❹ Evaluate $f(x_{n+1})$ and update data

Return best observed point.

Expected Improvement (EI) acquisition function

Let f_n^* be the best observed value so far.

Improvement at x :

$$I_n(x) = \max(f_n^* - f(x), 0).$$

Expected improvement :

$$\text{EI}_n(x) = \mathbb{E}[I_n(x)].$$

Since $f(x) \mid \mathcal{D}_n \sim \mathcal{N}(m_n(x), s_n^2(x))$, we have a closed-form expression : Annex

$$\text{EI}_n(x) = (f_n^* - m_n(x))\Phi(Z) + s_n(x)\phi(Z),$$

where Φ c.d.f. of $\mathcal{N}(0, 1)$ and $\phi = \Phi'$, and

$$Z = \frac{f_n^* - m_n(x)}{s_n(x)}.$$

Remark : EI automatically balances exploration via $s_n(x)$ and exploitation via $m_n(x)$.

Upper Confidence Bound (UCB)

Another acquisition function :

$$\text{UCB}_n(x) = m_n(x) + \kappa s_n(x)$$

(for minimization problems)

- $m_n(x)$: exploitation
- $s_n(x)$: exploration
- κ controls exploration level

Interpretation : Balance between promising mean and high uncertainty.

Exploration vs Exploitation

- Exploitation : evaluate near current best estimate
- Exploration : evaluate where uncertainty is high

GP provides both :

- Mean $m_n(x)$
- Variance $s_n^2(x)$

Acquisition functions combine them automatically.

Bayesian Optimization in Finance

Applications :

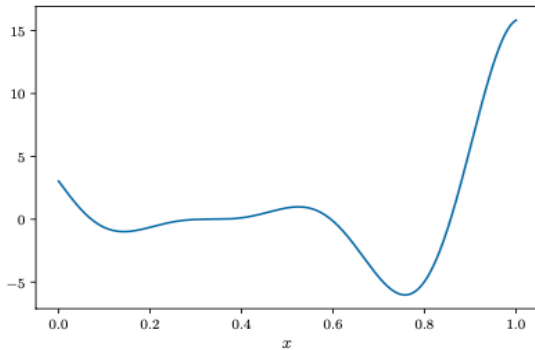
- Calibration of stochastic volatility models
- Tuning hedging strategy parameters
- Portfolio allocation under constraints
- XVA computation
- Optimal execution parameters

Advantage : Few expensive evaluations required.

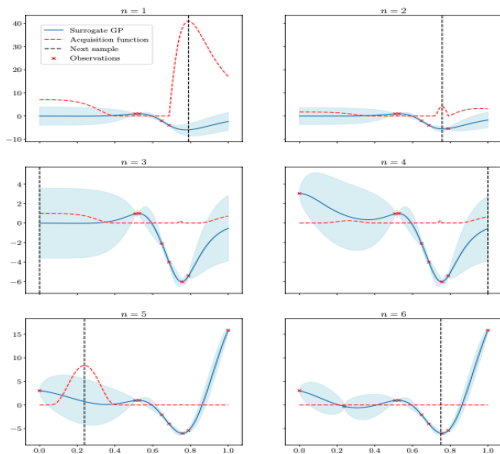
Illustration

Minimization of the objective function

$$f(x) = (6x - 2)^2 \sin(12x - 4).$$



Iterations of the Bayesian optimization



Remarks :

- Steps $n = 1, 2, 6$: exploitation stage (sampling where improvement is high)
- Steps $n = 3, 4, 5$: exploration stage (sampling where variance is high)

Summary of Gaussian Processes

- Bayesian non-parametric regression
- Closed-form posterior mean and variance
- Kernel encodes structural assumptions
- Hyperparameters learned by marginal likelihood
- Natural surrogate model for expensive functions

Advantages and limitations of GPs

Advantages :

- Non-parametric and flexible
- Provides uncertainty estimates for predictions
- Good for small to medium-sized datasets

Limitations ;

- Poor scalability ; $O(n^3)$ computational complexity
- Kernel selection and hyperparameter tuning can be challenging
- Struggles with very high-dimensional data

Outline

- 1 A primer on Gaussian process
 - Prediction
 - Bayesian optimization
- 2 Applications of GPR in finance
 - Applications to fast pricing and and fitting
 - GP surrogates for Delta Hedging
 - Other applications

Role of Gaussian Processes in Finance

Gaussian processes are used as **surrogate models** :

- Approximate expensive pricing functions
- Enable differentiable surrogates (fast Greeks / sensitivities)
- Interpolate sparse or noisy financial data
- Provide uncertainty quantification
- Facilitate optimization under uncertainty (BO)
- Enable fast optimization and calibration

GPs complement, rather than replace, financial models.

Applications to fast pricing and fitting

Offline phase :

- Generate a training set using a pricing model (FFT, PDE, MC)
- Store prices and sensitivities on a parameter grid
- Fit a Gaussian process surrogate

Online phase :

- Predict prices and Greeks instantly
- Negligible computational cost

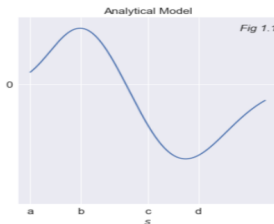
Key Insight

GP shifts computational complexity offline.

When training is completed, prediction is extremely fast !

Example to Gamma fitting

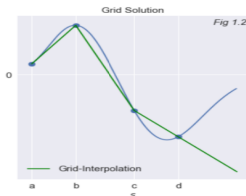
- Consider the position on a cliquet call option : study the Gamma $\Gamma(\cdot)$ as a function of the stock price S



Smooth representation of Γ (e.g. from pricing in a Heston model) for a range of stock prices requires a large computational workload !

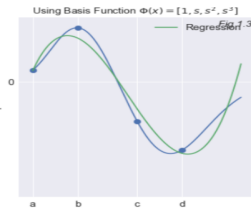
- Two classical approaches :

1. Grid solution : (i) calculate $\Gamma(S)$ for a selection of stock price $S \in \{a, b, c, d\}$, (ii) Value $\hat{\Gamma}(S)$ obtained via interpolation in the grid :

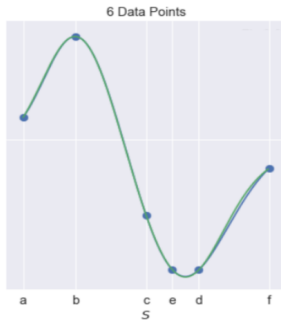


2. Regression model, e.g. :

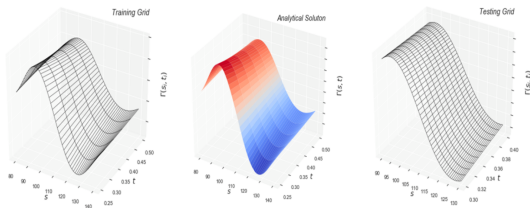
$$\hat{\Gamma}(S) = w_0 + w_1 S + w_2 S^2 + w_3 S^3.$$



- GPR fit for Γ :



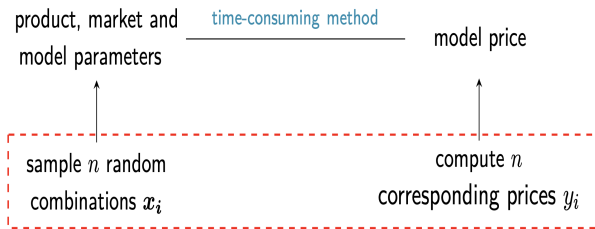
- Extension in two dimensions for training set with input : stock price S and maturity T
 - Training set with different combinations of (S_i, T_i)
 - Calculate analytical value Γ_i for each crossing point (S_i, T_i) of the grid
 - Once trained, GPR model gives values of Γ for unobserved combinations of (S, T)



Example : for $(S, T) = (110, 0.3)$, GPR gives -64.0186814394502 , while analytical value is -64.01868145719564 , hence an absolute error of $1.7745e - 08$.

Examples of derivative pricing

- Construct a **training set** :



- Fit a GPR model
- Fast prediction of new model prices : once trained, GP prediction is just a matrix-vector product.

Pricing European call options

GPR approach for predicting price even faster than Fast Fourier Transform (FFT)-based European call option prices in Heston [Annex](#) and Variance Gamma (VG) models.

• Training set

	Product/market	VG	Heston
Training set	K : 40% $\rightarrow$ 160%	σ : 0.05 $\rightarrow$ 0.45	κ : 1.4 $\rightarrow$ 2.6
	T : 11M $\rightarrow$ 1Y	ν : 0.55 $\rightarrow$ 0.95	ρ : -0.85 $\rightarrow$ -0.55
	r : 1.5% $\rightarrow$ 2.5%	θ : -0.35 $\rightarrow$ -0.05	θ : 0.45 $\rightarrow$ 0.75
	q : 0% $\rightarrow$ 5%		η : 0.01 $\rightarrow$ 0.1
			v_0 : 0.01 $\rightarrow$ 0.1
Test set	K : 50% $\rightarrow$ 150%	σ : 0.1 $\rightarrow$ 0.4	κ : 1.5 $\rightarrow$ 2.5
	T : 11M $\rightarrow$ 1Y	ν : 0.6 $\rightarrow$ 0.9	ρ : -0.8 $\rightarrow$ -0.6
	r : 1.5% $\rightarrow$ 2.5%	θ : -0.3 $\rightarrow$ -0.1	θ : 0.5 $\rightarrow$ 0.7
	q : 0% $\rightarrow$ 5%		η : 0.02 $\rightarrow$ 0.1
			v_0 : 0.02 $\rightarrow$ 0.1

Table 1: Parameter ranges used for training and validation.

→ Sample n values of each parameter

- For each parameter combination, prices $\mathbf{P} = (P_i^{FFT})$ of European Call option is calculated by FFT $\rightarrow$ Training set $(\mathbf{x}, \mathbf{P}) \rightarrow$ algorithm to fit GPR model
- Pricing performance measured in terms of maximum and average absolute errors :

$$MAE = \max \{ P_i^{FFT} - P_i^{GPR}, i = 1, \dots, n \}$$

$$AAE = \frac{1}{n} \sum_{i=1}^n |P_i^{FFT} - P_i^{GPR}|$$

for both training set (in-sample) and test set (out-of-sample).

Size of training set	VG			Heston		
	5000	10 000	20 000	5000	10 000	20 000
In-sample prediction						
MAE	0.0065	0.0051	0.0046	0.0078	0.0086	0.0088
AAE	3.2771e-04	2.5847e-04	2.0492e-04	5.3348e-04	4.3611e-04	3.6575e-04
Out-of-sample prediction						
MAE	0.0028	0.0022	0.0016	0.0060	0.0054	0.0048
AAE	2.2508e-04	1.6942e-04	1.2828e-04	5.8991e-04	4.4112e-04	3.6623e-04
Speed-up	$\times 30$	$\times 15$	$\times 7$	$\times 40$	$\times 20$	$\times 10$

Table 2: GPR performance with respect to the FFT-method, for differently sized training sets. Out-of-sample predictions correspond to a test set of size 100 000. Speed-ups are obtained by comparing the CPU time for both methods.

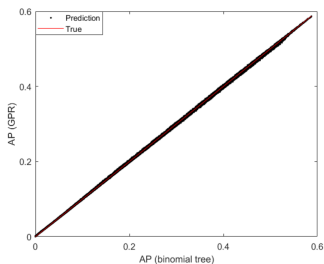
Barrier options

- Down-and-out-barrier put option (DOPB) in Heston model with barrier level H , strike K , and maturity T with

$$H \in [0, 55, 0.99]$$

- Prices computed by Monte-Carlo (100 000 price paths of underlying, and daily steps 1/250 up to maturity date)

Out-of-sample performance



MAE 0.0086

AAE 9.1684e-04

Speed-up ×70

→ model trained on 10 000 points, tested on 100 000 points.

Option price and Delta Hedging surface

- European option price/Delta Hedge in a BS or local volatility model :

$$P(t, S) = \mathbb{E}^{\mathbb{Q}}[e^{-r(T-t)}g(S_T)|S_t = S], \quad \Delta(t, S) = \frac{\partial P(t, S)}{\partial S}.$$

- We consider a situation where $P(t, S)$ is not directly known. Instead : a training set $\mathcal{D} = \{(t_i, S_i, Y_i) : i = 1, \dots, n\}$ where $Y_i \simeq P(t_i, S_i)$, and we aim to learn $(t, S) \mapsto \Delta(t, S)$.
 - Pricing model but the exact computation of $P(t, S)$ is time-consuming $\rightarrow \mathcal{D}$ is a collection of inputs with exact value $Y_i = P(t_i, S_i)$.
 - Option prices evaluated via Monte-Carlo $\rightarrow \mathcal{D}$ collection of noisy samples Y_i of $P(t_i, S_i)$.

Surrogate gradients

- Aim : provide an estimate of $\Delta(t, S)$ for arbitrary $(t, S) \in \mathcal{D}'$ in some test set $\mathcal{D}'$. This could include
 - *in-sample* prediction, i.e., for some $(t, S) \in \mathcal{D}$,
 - *ou-of-sample* predictions, by extrapolating (t, S) outside the convex hull of $\mathcal{D}$, e.g. for $t > T_0$ if the training was confined to $t < T_0$.
 - Learning task in two steps :
 - 1 Fit a statistically driven mapping $(t, S) \mapsto \hat{P}(t, S)$ from $\mathcal{D}$, called *surrogate* model.
 - 2 Set $\hat{\Delta}(t, S) = \frac{\partial \hat{P}(t, S)}{\partial S}$. This is done analytically :

Differentiate the surrogate, not the Monte Carlo.
-
- Error in Δ is reduced since only a single approximation is used, and the differentiation is exact.
 - Uncertainty quantification on Δ from the uncertainty in $\hat{P}$.

GP for surrogate

- We use a GPR for learning the option price surface $x = (t, S) \mapsto P(x)$ with an observation model :

$$Y(x) = P(x) + \varepsilon,$$

with a Gaussian noise $\varepsilon \sim \mathcal{N}(0, \sigma_\varepsilon^2)$. Recall that the posterior mean and covariance prediction of $P(x^*)$ for an input x^* is

$$\begin{aligned}\hat{m}(x^*) &= K(x^*, \mathbf{x})[K(\mathbf{x}, \mathbf{x}) + \sigma_\varepsilon^2 I_n]^{-1} \mathbf{y} \\ \hat{C}_P(x^*, x^{*'}) = \text{cov}(P(x^*), P(x^{*'})) &= k(x^*, x^{*'}) - K(x^*, \mathbf{x})[K(\mathbf{x}, \mathbf{x}) + \sigma_\varepsilon^2 I_n]^{-1} K(\mathbf{x}, x^{*'}),\end{aligned}$$

where k is the GP kernel, and $\mathbf{x} = (x_1, \dots, x_n)$, $\mathbf{y} = (y_1, \dots, y_n)$, with $(x_i, y_i) \in \mathcal{D}$.

Obtaining the greeks

From the GP fitted model for $P \sim \mathcal{N}(\hat{m}, \hat{C}_P)$, the gradient of P forms another GP : $\nabla_x P \sim \mathcal{N}(\hat{m}_g, \hat{C}_g)$ with mean and covariance predictor

$$\hat{m}_g(x^*) = \frac{\partial \hat{m}}{\partial x}(x^*) = \frac{\partial K}{\partial x}(x^*, \mathbf{x})[K(\mathbf{x}, \mathbf{x}) + \sigma_\varepsilon^2 I_n]^{-1} \mathbf{y}$$

$$\hat{C}_g(x^*, x^{*'}) = \frac{\partial^2 \hat{C}_P}{\partial x \partial x'} = \frac{\partial^2 k}{\partial x \partial x'}(x^*, x^{*'}) - \frac{\partial K}{\partial x}(x^*, \mathbf{x})[K(\mathbf{x}, \mathbf{x}) + \sigma_\varepsilon^2 I_n]^{-1} \frac{\partial K}{\partial x'}(\mathbf{x}, x^{*'}).$$

→ This yields the posterior variance $\hat{V}_g(x^*) = \hat{C}_g(x^*, x^*)$ of $\hat{m}_g(x^*)$, hence a confidence interval for $\nabla_x P$.

Toy illustration for the call option in BS model

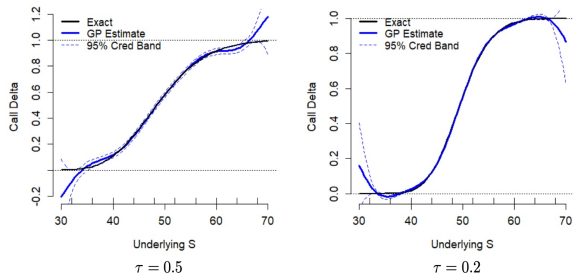


Figure 1. Estimated Delta $\hat{\Delta}$ for a Black-Scholes Call. Left panel extrapolates for longer maturity $\tau = 0.5$; right panel is an in-sample test set $\tau = 0.2$. We also show the 95% credible bands from (11).

Test in a local volatility model

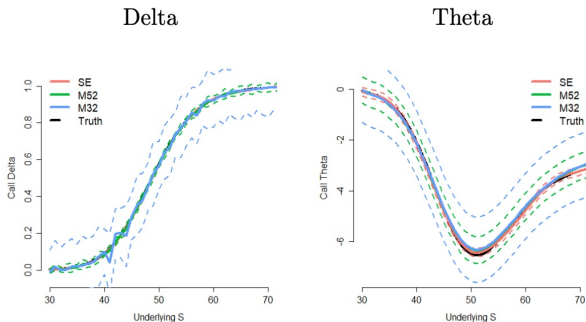


Figure – Fitted greeks vs Benchmark for three different kernels

Implied Volatility Surface Modeling

Problem :

- Implied volatility is noisy and sparse across strikes and maturities
- Classical interpolation may violate arbitrage constraints

How GPs help :

- Smooth interpolation across (K, T)
- Flexible non-parametric modeling
- Robustness to noise
- Provides uncertainty bands on implied volatility

Modeling :

- Inputs : Strike price K and time to maturity T
- Output : Implied volatility $\sigma(K, T)$

Application : Model Calibration

Goal :

$$\min_{\theta} \sum_i (P^{model}(\theta; K_i, T_i) - P_i^{market})^2$$

Challenge : Each evaluation of P^{model} is expensive.

GP-based approach :

- Learn surrogate $\theta \mapsto P(\theta)$
- Use Bayesian optimization for calibration
- Obtain uncertainty on parameters

Application : Risk Measures

Portfolio loss :

$$L(x) = f(\text{risk factors})$$

Monte Carlo estimation of VaR / CVaR is expensive.

GP surrogate :

- Approximate $L(x)$
- Focus sampling near tail events
- Fast estimation of risk measures

Uncertainty quantification is crucial for regulation.

Multi-Fidelity Pricing with Gaussian Processes

Idea :

- Combine cheap approximations (coarse MC)
- With expensive high-accuracy simulations

Model :

$$f_H(x) = \rho f_L(x) + \delta(x)$$

where

- $f_H(x)$: high-fidelity (accurate, expensive) model
- $f_L(x)$: low-fidelity (cheap, biased) approximation : captures global structure
- $\delta(x)$: discrepancy between fidelities : corrects systematic bias

Use multi-output GP to fuse information.

- Significant reduction in simulation cost
- Widely used in XVA computations

Application : American Options

Pricing via optimal stopping :

$$V_t = \max\{g(S_t), \mathbb{E}[V_{t+1} \mid S_t]\}$$

Classical method : Longstaff–Schwartz (polynomial regression)

GP-based approach :

- GP regression for continuation value
- Uncertainty near exercise boundary
- Improved stability

When Not to Use Gaussian Processes

- Very high-dimensional inputs (> 30 – 50)
- Massive datasets ($n > 50,000$)
- Non-smooth or discontinuous payoffs

Alternative approaches :

- Deep neural networks
- Random forests

Conclusion and Bridge to Lecture 2

Lecture 1 takeaway : Gaussian processes

- probabilistic surrogate modeling for pricing, Greeks, and calibration
- uncertainty quantification as a first-class output
- Bayesian optimization for expensive objectives
- Many financial applications have been recently developed in literature, see list of projects.

Key limitation :

- scalability in high dimension and with large datasets

Next : Lecture 2

- deep learning for high-dimensional stochastic control
- applications to deep hedging

References

- ① C. E. Rasmussen and C. Williams : Gaussian process for machine learning, 2006, MIT press, [online version](#)
- ② M. Ludkovski, J. Risk : Gaussian processes for quantitative finance, 2025, SpringerBriefs
- ③ P.E. Frazier : A tutorial on Bayesian optimization, [arXiv: 1807.02811](#)
- ④ J. De Spiegeleer, D. Madan, S. Reyners, W. Schoutens : "Machine learning for quantitative finance : fast derivative pricing, hedging and fitting", 2018, *Quantitative Finance*.
- ⑤ J. Gonzalvez, E. Lezmi, T. Roncalli, J. Xu : "Financial applications of Gaussian processes and Bayesian optimization", [arXiv:1903.04841v1](#)

Let (X_1, X_2) a Gaussian vector :

$$\begin{bmatrix} X_1 \\ X_2 \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} m_1 \\ m_2 \end{bmatrix}, \begin{bmatrix} \Sigma_1 & \Sigma_{21}^\top \\ \Sigma_{21} & \Sigma_2 \end{bmatrix}\right).$$

so that $X_i \sim \mathcal{N}(m_i, \Sigma_i)$, $i = 1, 2$, and $\Sigma_{21} = \text{cov}(X_2, X_1)$. Then

$$X_2 | X_1 = x_1 \sim \mathcal{N}(m_{2|1}, \Sigma_{2|1})$$

where

$$\begin{aligned} m_{2|1} &= \mathbb{E}[X_2 | X_1 = x_1] &= m_2 + \Sigma_{21} \Sigma_1^{-1} (x_1 - m_1) \\ \Sigma_{2|1} &= \text{Var}[X_2 | X_1 = x_1] &= \Sigma_2 - \Sigma_{21} \Sigma_1^{-1} \Sigma_{21}^\top. \end{aligned}$$

Annex

back

For $X \sim \mathcal{N}(\mu, \sigma^2)$, we have

$$\mathbb{E}[\max(c - X, 0)] = (c - \mu)\Phi\left(\frac{c - \mu}{\sigma}\right) + \sigma\phi\left(\frac{c - \mu}{\sigma}\right).$$

where Φ c.d.f. of $\mathcal{N}(0, 1)$ and $\phi = \Phi'$.

$$\begin{aligned}dS_t &= S_t(rdt + \sqrt{\vartheta_t}dW_t) \\d\vartheta_t &= \kappa(\eta - \vartheta_t)dt + \theta\sqrt{\vartheta_t}dB_t, \quad \vartheta_0 = \nu_0,\end{aligned}$$

with $d < W, B > = \rho dt$, and $2\kappa\eta > \theta^2$.